[Name of Institution]
Department of [Department Name]

Database Management Systems — Course Project Report

Bus Factor Tracker

Estimating Open-Source Maintainer Burnout and Bus Factor Risk from Real GitHub Data

---

Submitted by

Dhakshin	CB.SC.U4AIE24313
Rohith Ravi	CB.SC.U4AIE24350
Sanjay Kumar S	CB.SC.U4AIE24354
Akhilan	CB.SC.U4AIE24362

Semester V • October 2026

Abstract

Open-source software forms the base layer of almost every modern application, yet much of it is maintained by a handful of volunteers. When such a maintainer burns out or leaves, every project that depends on their work is exposed, often with no warning. The *bus factor* of a project, the number of people who would have to disappear before it stalls, is a simple measure of this risk.

This report presents the **Bus Factor Tracker**, a database-centred system that estimates bus factor and maintainer-burnout risk for real GitHub repositories from public data only. A Python sync engine pulls repository metadata, per-contributor commit statistics, weekly activity, issues, pull requests and releases from the GitHub REST API into a MySQL 8.0 database of nine relations in Boyce–Codd normal form. Each repository is written inside a single ACID transaction. Bus factor and median issue-response time are computed in SQL with common table expressions and window functions, and three burnout signals (commit-activity decline, inactivity of the top contributor, and slow issue responses) adjust a base risk level derived from the bus factor. A view, a stored function, a stored procedure and a trigger expose the latest risk state, detect declining repositories and record every risk increase. A FastAPI service and a React dashboard present the results with a plain-English explanation of each score.

Applied to 42 widely used repositories (17,306 contributors, 16,490 pull requests, 6,211 issues), the system found that 17 repositories (40%) have a bus factor of 1. Adding the burnout signals moved 9 repositories up at least one risk level, each change being logged automatically by the trigger. All read endpoints of the API respond in under 10 ms. The report also discusses the limitations observed on real data, such as automation bots appearing as top contributors, and the design decisions they motivate.

Keywords: bus factor, truck factor, open-source sustainability, maintainer burnout, relational database, MySQL, window functions, triggers, GitHub REST API.

Contents

1 Introduction **5**

1.1 Background . . . 5
1.2 The bus factor . . . 5
1.3 Problem statement . . . 5
1.4 Objectives . . . 6
1.5 Scope . . . 6
1.6 Related work . . . 6
1.7 Organisation of the report . . . 7

2 Methodology **8**

2.1 System architecture . . . 8
2.2 Data collection . . . 8
2.3 Database design . . . 9
2.3.1 Entity–relationship model . . . 9
2.3.2 Relational schema . . . 11
2.3.3 Normalisation . . . 11
2.3.4 Indexes . . . 12
2.4 Transaction design . . . 12
2.5 Computing the measurements in SQL . . . 13
2.5.1 Bus factor . . . 13
2.5.2 Median issue-response time . . . 13
2.5.3 Burnout signals . . . 13
2.6 Risk scoring . . . 14
2.7 Database objects . . . 15
2.8 Application layer . . . 16

3 Results and Discussion **17**

3.1 Experimental setup . . . 17
3.2 Data collected . . . 17
3.3 Risk distribution . . . 17
3.4 Effect of the burnout signals . . . 19
3.5 Case studies and validation . . . 19
3.6 Performance . . . 21
3.7 Per-repository results . . . 22

3.8 Discussion of limitations 23
3.9 Challenges and lessons learned 23

4 Conclusion and Future Work **25**
4.1 Conclusion 25
4.2 Future work 25

References **26**

List of Figures

2.1 System architecture and data flow. 8
2.2 Entity–relationship diagram. Underlined attributes form the primary key; italic attributes are foreign keys. Solid lines are mandatory relationships, dashed lines optional ones (a deleted GitHub account leaves the author unknown). Risk_Alert also references Repository (not drawn). 10

3.1 Risk level of the 42 repositories (latest snapshot). 18
3.2 Bus factor versus top-contributor share for the 42 repositories. 18
3.3 Dashboard: summary counts per risk level and the recent risk increases recorded by the trigger. 20
3.4 Repository detail for `pallets/flask`: the risk level with its reasons and the burnout signals. 21

List of Tables

2.1 GitHub REST API endpoints used per repository. 8
2.2 Relations, keys and purpose. 11
2.3 Normal forms as applied to the schema. 12
2.4 Risk levels and burnout thresholds. 15

3.1 Rows stored per relation after the full sync. 17
3.2 Risk levels with average bus factor and top-contributor share. 18
3.3 Average bus factor by primary language (languages with ≥ 2 repositories). 19
3.4 Risk increases recorded by `trg_snapshot_risk_alert`. 19
3.5 Median API latency over 20 requests. 21
3.6 Latest snapshot per repository. BF = bus factor; Top = top contributor's share of commits; ΔAct = commit-activity change; Inact. = days since the top contributor's last commit; "–" = not enough data. 22

Chapter 1

Introduction

1.1 Background

Open-source libraries are reused at enormous scale: a single web application can pull in hundreds of transitive dependencies, each maintained by people the application's developers have never met. Studies of the open-source ecosystem describe this as critical public infrastructure that is largely maintained without pay [1, 2]. The consequences of a single maintainer's decision became widely visible in March 2016, when the removal of the eleven-line `left-pad` package from the npm registry broke the builds of thousands of downstream projects [3].

Less dramatic versions of the same failure happen continually. Maintainers lose interest, change jobs or simply burn out, and projects that looked healthy from the outside stop receiving fixes. Research on failed open-source projects lists lack of maintainer time and interest among the most common reasons a project is abandoned [4], and toxic or demanding interactions with users are a documented source of maintainer stress [5].

1.2 The bus factor

The *bus factor* (also called the *truck factor*) of a software project is the minimum number of developers who would have to leave the project ("be hit by a bus") before it can no longer be maintained. A bus factor of 1 means that a single person holds knowledge or responsibility the project cannot function without. Several algorithms have been proposed to estimate it from version-control history, including file-authorship approaches [6] and commit-based approaches applied directly to Git repositories [7]. Practitioners report that the metric is useful but that raw estimates need context to be acted on [8].

1.3 Problem statement

Developers choosing a dependency, and organisations funding open-source work, have no simple way to see which projects are one person away from abandonment. Existing tools either show raw activity graphs for a single repository, focus on security practices, or require a heavyweight analytics platform (Section 1.6). The problem addressed in this project is:

Design and build a database-backed system that estimates the bus factor and maintainer-burnout risk of real GitHub repositories from public data, explains each estimate, and tracks how the risk changes over time.

1.4 Objectives

1. Collect real repository, contributor, issue, pull-request and release data from the GitHub REST API — no simulated data.
2. Design a normalised relational schema (BCNF) with full referential integrity, suitable for both current-state queries and time-series history.
3. Compute the bus factor, median issue-response time and burnout signals inside the database using SQL (CTEs and window functions).
4. Use DBMS features where they fit the problem: transactions for consistent syncs, a view, a stored function, a stored procedure and a trigger.
5. Combine the measurements into an explainable risk level per repository, and record every increase in risk.
6. Present the results through a REST API and an interactive web dashboard.

1.5 Scope

The system tracks a configurable list of public GitHub repositories (42 in the evaluation, chosen to mix large, well-staffed projects with small single-maintainer utilities and one archived project). It relies on the data GitHub exposes through its public API with a personal access token. It does not analyse source code, mailing lists or chat channels, and it does not attempt per-file authorship analysis.

1.6 Related work

Truck-factor research. Avelino et al. [6] estimate truck factors from file authorship (degree-of-authorship) and validate the estimates with developers of popular GitHub projects. Cosentino et al. [7] compute bus-factor indicators for Git repositories. Jabrayilzade et al. [8] study how the bus factor is used in industry. These works analyse repositories at a point in time; they do not maintain a continuously updated history or alert on change.

Community-health tooling. The CHAOSS project [9] defines a broad catalogue of open-source community-health metrics and provides analytics platforms to compute them. The OpenSSF Scorecard [10] assesses security-related practices of a repository, including whether it has been recently active. GitHub's own repository Insights pages show activity graphs for one repository at a time.

This project. The Bus Factor Tracker sits between these: it is a lightweight, single-purpose system that (i) combines commit concentration with burnout signals into one explainable risk level, (ii) ranks many repositories side by side, and (iii) keeps an append-only history so that declines and risk increases are detected inside the database itself.

1.7 Organisation of the report

Chapter 2 describes the system architecture, data collection, database design, SQL techniques and risk-scoring method. Chapter 3 presents the results on 42 real repositories and discusses them, including performance, validation cases and limitations. Chapter 4 concludes and outlines future work.

Chapter 2

Methodology

2.1 System architecture

The system is split into four modules connected by the database (Figure 2.1). Collecting data is slow (tens of seconds of API calls per repository) while reading results must be fast, so the two are deliberately separated: the sync engine writes, and the API almost only reads.

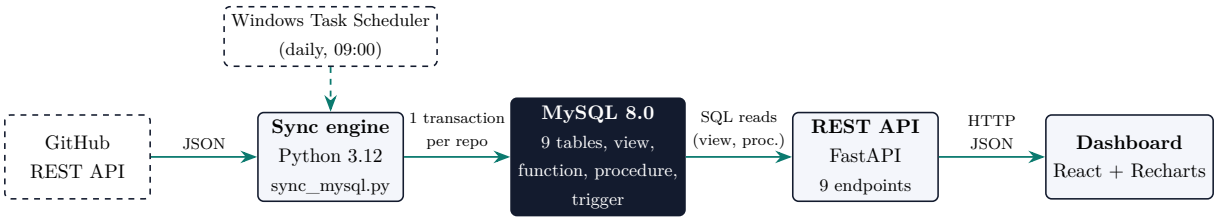


Figure 2.1: System architecture and data flow.

2.2 Data collection

The sync engine (`scripts/sync_mysql.py`) reads the list of tracked repositories from `repos.txt` and, for each repository, calls the GitHub REST API endpoints in Table 2.1 [11].

Table 2.1: GitHub REST API endpoints used per repository.

Endpoint	Data obtained
`/repos/{o}/{r}`	Language, stars, forks, creation date, archived flag
`/repos/{o}/{r}/stats/contributors`	Per-contributor commit, addition and deletion counts, broken down by week
`/repos/{o}/{r}/stats/commit_activity`	Total commits per week for the last 52 weeks
`/repos/{o}/{r}/issues`	Issues (state, opened/closed time, author); pull requests filtered out
`/repos/{o}/{r}/issues/{n}/comments`	First comment on an issue, giving the first-response time
`/repos/{o}/{r}/pulls`	Pull requests (state, opened/merged time, author)
`/repos/{o}/{r}/releases`	Releases (tag, publish time, author)

Several design choices keep the sync within GitHub's limit of 5,000 authenticated requests per hour and make it robust:

- **Pre-aggregated statistics.** Contributor and weekly-activity statistics come from GitHub's `stats` endpoints, one call each, instead of one call per commit.
- **Incremental sync.** The repository's `last_synced_at` timestamp is read before each sync; issues are requested with GitHub's `since` parameter, and pull-request and release pagination stops as soon as it reaches items older than the last sync.
- **Capped work.** Pagination is capped at three pages, and the first-response lookup is limited to the 30 most recently updated issues with comments; those lookups run concurrently (10 threads).
- **Lazy statistics.** GitHub computes `stats` endpoints in the background and answers `202 Accepted` until they are ready. The engine retries with capped exponential back-off (2, 4, 8, 10, 10 s). If `commit_activity` never becomes ready, weekly totals are derived by summing the per-week counts already returned by `stats/contributors`.
- **Network resilience.** Dropped connections, truncated responses and time-outs are retried up to three times per request.
- **No network I/O inside a transaction.** All API calls for a repository complete before its database transaction begins, so slow responses never hold locks.

A Windows Task Scheduler job runs the sync daily, so that each repository gains one snapshot per day and trends become measurable over time.

2.3 Database design

2.3.1 Entity–relationship model

The conceptual model (Figure 2.2) follows Chen's entity–relationship approach [12]. Repository and Contributor are the two strong entities. Their many-to-many relationship carries its own attributes (commit counts and first/last commit dates), so it is modelled as the associative entity Repo_Contributor. Weekly_Activity is a weak entity identified by its repository and week. Snapshots and alerts form an append-only history.

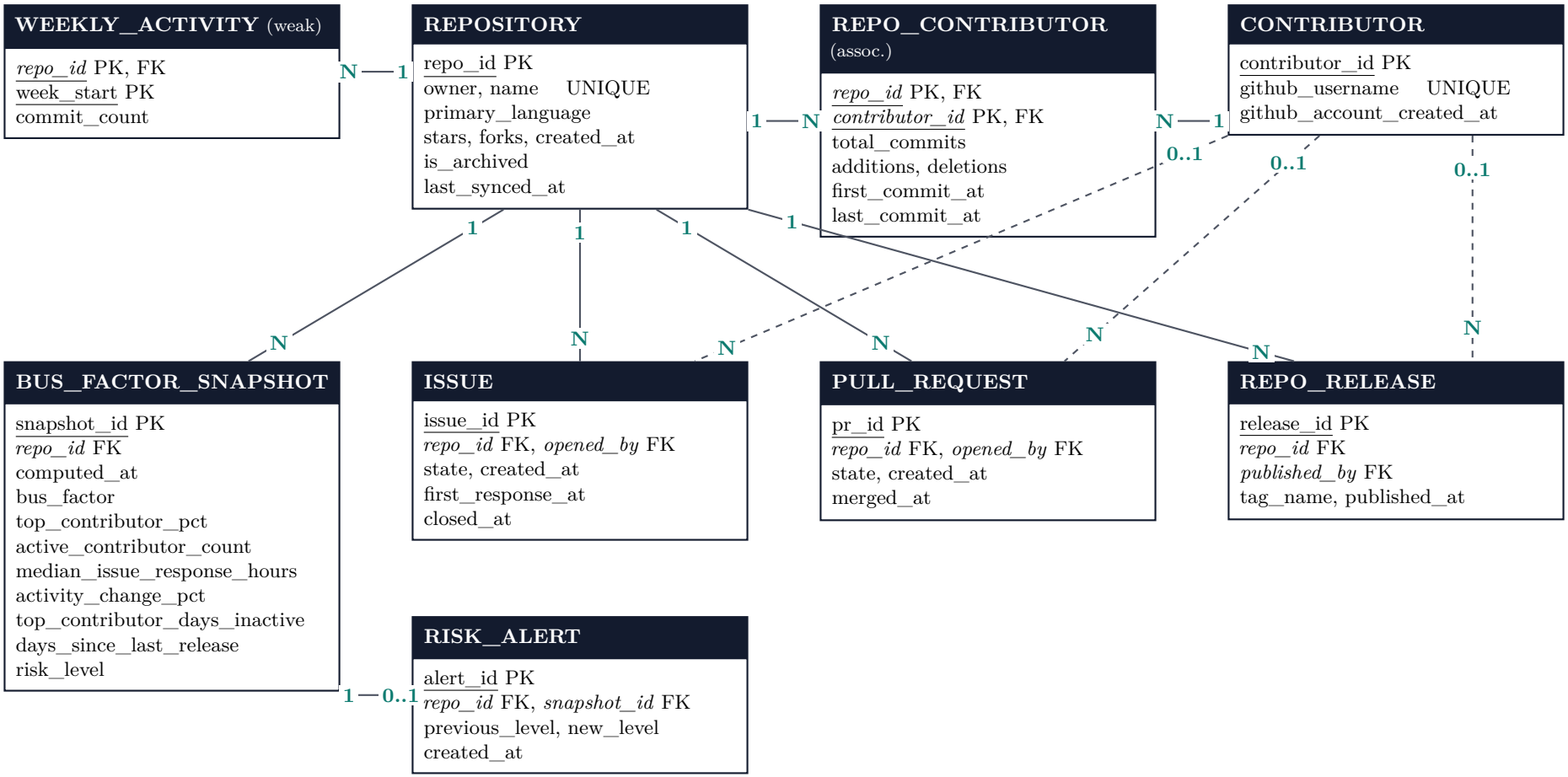


Figure 2.2: Entity–relationship diagram. Underlined attributes form the primary key; italic attributes are foreign keys. Solid lines are mandatory relationships, dashed lines optional ones (a deleted GitHub account leaves the author unknown). Risk_Alert also references Repository (not drawn).

2.3.2 Relational schema

The ER model maps to nine relations (Table 2.2). GitHub's own identifiers are used as primary keys for issues, pull requests and releases (`BIGINT`), which makes re-syncing idempotent and guarantees that keys never collide across repositories. `release` is a reserved word in MySQL, hence `repo_release`.

Table 2.2: Relations, keys and purpose.

Relation	Primary key / foreign keys	Purpose
`repository`	repo_id; UNIQUE(owner, name)	One row per tracked repository
`contributor`	contributor_id; UNIQUE(github_username)	People, shared across repositories
`repo_contributor`	(repo_id, contributor_id)	Associative entity: commits per person per repository
`issue`	issue_id; FK repo_id, opened_by	Issues with first-response time
`pull_request`	pr_id; FK repo_id, opened_by	Pull requests
`repo_release`	release_id; FK repo_id, published_by	Releases
`weekly_activity`	(repo_id, week_start)	Weak entity: commits per week
`bus_factor_snapshot`	snapshot_id; FK repo_id	Append-only time series of computed risk
`risk_alert`	alert_id; FK repo_id, snapshot_id	Written by a trigger when risk increases

Referential integrity. Every child relation references `repository(repo_id)` with `ON DELETE CASCADE`. Removing a repository from the dashboard is therefore a single `DELETE` statement, and the DBMS removes its contributor statistics, issues, pull requests, releases, weekly activity, snapshots and alerts in the same operation. Contributors are shared between repositories, so they are deliberately not cascaded.

2.3.3 Normalisation

All relations are in Boyce–Codd normal form: for every non-trivial functional dependency $X \to Y$, X is a superkey [13–15]. Table 2.3 shows how each normal form is reflected in the design.

Table 2.3: Normal forms as applied to the schema.

Normal form	Application
1NF	Contributors of a repository are rows in `repo_contributor`, not a comma-separated list; weekly activity is one row per week, not an array.
2NF	In `repo_contributor`, `total_commits` depends on the whole key (`repo_id`, `contributor_id`); `github_username` depends on `contributor_id` alone and therefore lives in `contributor`.
3NF	`risk_level` is derived from the bus factor and burnout signals and changes on every sync, so it is stored on the snapshot that produced it, never on `repository`.
BCNF	`repository` has two candidate keys, `repo_id` and (`owner`, `name`); both determine all other attributes. The same holds for `contributor`.

Deliberate redundancy. `bus_factor_snapshot` stores values that could in principle be recomputed from other relations. This is intentional: the source relations are overwritten on each sync, so a snapshot is the only record of what the measurements *were* at `computed_at`.

2.3.4 Indexes

Besides primary keys and unique constraints, secondary indexes support the most frequent access paths: `(repo_id, computed_at)` on snapshots for "latest snapshot" and trend queries, `repo_id` on `repo_contributor` and `issue` for per-repository aggregation, and `created_at` on `risk_alert` for the newest-first alert feed.

2.4 Transaction design

Each repository is synchronised in one transaction (Listing 2.1), following the atomicity and durability guarantees of ACID transactions [16]. If any step fails, such as a network error during the preceding fetch, a malformed row or a lost database connection, the whole repository rolls back. The previous snapshot then remains the latest one, so the dashboard never shows a score computed from half-updated data. Every write except the snapshot is an `INSERT ... ON DUPLICATE KEY UPDATE`, making repeated syncs idempotent.

Listing 2.1: Per-repository transaction (simplified from `sync_repo`).

```
cur = conn.cursor() # conn.autocommit = False
try:
    repo_id = upsert_repo(cur, owner, repo, meta)
    for entry in contributor_stats:
        upsert_repo_contributor(cur, repo_id, ...)
    upsert_weekly_activity(cur, repo_id, commit_activity)
    for issue in issues: upsert_issue(cur, repo_id, ...)
    for pr in pull_requests: upsert_pull_request(cur, repo_id, ...)
    for release in releases: upsert_release(cur, repo_id, ...)
    compute_and_insert_snapshot(cur, repo_id) # fires the trigger
    conn.commit()
except Exception:
    conn.rollback() # nothing for this repository is kept
```

```
    raise
```

2.5 Computing the measurements in SQL

2.5.1 Bus factor

The bus factor is computed as the smallest number of top contributors who together authored at least 50% of all commits. Listing 2.2 does this in a single pass with a common table expression and window functions: contributors are ranked by commits, a running total is accumulated in that order, and the first rank at which the running total reaches half the repository's total is the bus factor.

Listing 2.2: Bus factor, top-contributor share and contributor count.

```
WITH ranked_contributors AS (
    SELECT contributor_id, total_commits,
           SUM(total_commits) OVER () AS repo_total_commits,
           SUM(total_commits) OVER (ORDER BY total_commits DESC) AS running_total,
           ROW_NUMBER() OVER (ORDER BY total_commits DESC) AS contributor_rank,
           COUNT(*) OVER () AS contributor_count
    FROM repo_contributor
    WHERE repo_id = %s
)
SELECT MIN(CASE WHEN running_total >= 0.5 * repo_total_commits
                THEN contributor_rank END) AS bus_factor,
       MAX(CASE WHEN contributor_rank = 1
                THEN ROUND(100 * total_commits / repo_total_commits, 1) END)
                                                        AS top_contributor_pct,
       MAX(contributor_count) AS contributor_count
FROM ranked_contributors;
```

2.5.2 Median issue-response time

`PERCENTILE_CONT` is only available from MySQL 8.0.28 [17], so the median time to the first comment on an issue uses a portable window-function median: response times are numbered with `ROW_NUMBER()`, counted with `COUNT(*) OVER ()`, and the middle one or two rows are averaged.

2.5.3 Burnout signals

Three signals are computed for each snapshot:

Commit-activity change. Using conditional aggregation over `weekly_activity`, the average commits per week in the most recent 12 weeks is compared with the average over the 40 weeks before:

$$\Delta_{\text{act}} = 100 \times \frac{\overline{c}_{\text{last 12 wk}} - \overline{c}_{\text{prev 40 wk}}}{\overline{c}_{\text{prev 40 wk}}} \ \%.$$

Top-contributor inactivity. Days since the last commit of the contributor with the most commits, from `repo_contributor.last_commit_at` (derived from the last week with a

non-zero commit count).

Slow issue response. The median issue-response time described above.

Days since the latest release is also stored, but only as context: many healthy libraries never publish GitHub releases.

2.6 Risk scoring

Algorithm 1 combines the measurements. The bus factor sets a base level; an archived repository is Critical outright; and each independent burnout signal raises the level by one, capped at Critical. The function returns the level together with a list of plain-English reasons. The same function is used by the sync engine (to store the level) and by the API (to explain it), so the explanation always matches the stored score.

Algorithm 1 Risk assessment (`assess_risk`)

Require: bus factor b, top-contributor share s (%), median response m (h), activity change a (%), top-contributor inactivity d (days), archived flag

Ensure: risk level $L \in \{0, 1, 2, 3\}$ (Healthy, Medium, High, Critical) and reasons R

$L \leftarrow 3$ **if** $b = 1$ **else** 2 **if** $b = 2$ **else** 1 **if** $b \leq 4$ **else** 0
$R \leftarrow$ [explanation of b and s]
if archived **then**
    $L \leftarrow 3$; add "repository is archived" to R
end if
if $m > 720$ **then**
    $L \leftarrow L + 1$; add "slow issue responses" to R
end if
if $a \leq -50$ **then**
    $L \leftarrow L + 1$; add "commit activity down" to R
end if
if $d > 180$ **and** $s \geq 25$ **then**
    $L \leftarrow L + 1$; add "top contributor inactive" to R
end if
return $(\min(L, 3),\ R)$

Table 2.4: Risk levels and burnout thresholds.

Component	Condition	Effect
Bus factor	1 / 2 / 3–4 / ≥ 5	Critical / High / Medium / Healthy
Archived repository	`is_archived = TRUE`	Critical
Slow issue responses	median first reply > 30 days	+1 level
Activity decline	last 12 weeks $\geq 50\%$ below previous 40	+1 level
Top contributor inactive	no commit > 180 days *and* share $\geq 25\%$	+1 level
Release recency	days since last release	shown only

The 25% share condition on the inactivity signal was added after testing on real data (Section 3.5).

2.7 Database objects

View. `v_repo_latest_snapshot` joins every repository to its newest snapshot (a `LEFT JOIN`, so repositories without a snapshot still appear). Selecting the newest row by `snapshot_id` rather than `MAX(computed_at)` avoids duplicate rows when two snapshots share a timestamp. All API endpoints that need "current state" read from this view.

Stored function. `fn_risk_rank(level)` maps Healthy, Medium, High and Critical to 1–4. The `ENUM` stores the levels in the opposite order, so this function is the single definition of "worse than", used by the trigger, the procedure and the API's risk ordering.

Stored procedure. `sp_declining_repos(p_days)` pairs each repository's latest snapshot with the newest snapshot at least `p_days` older and returns repositories whose bus factor fell or whose risk level rose.

Trigger. `trg_snapshot_risk_alert` (Listing 2.3) fires after every snapshot insert, compares the new risk level with the repository's previous snapshot, and writes a row to `risk_alert` when the risk has increased. Because it runs inside the sync transaction, a rolled-back sync can never leave an alert pointing at a snapshot that does not exist.

Listing 2.3: Trigger recording risk increases.

```
CREATE TRIGGER trg_snapshot_risk_alert
AFTER INSERT ON bus_factor_snapshot
FOR EACH ROW
BEGIN
    DECLARE v_prev_level VARCHAR(10);
    SET v_prev_level = (
        SELECT s.risk_level FROM bus_factor_snapshot s
        WHERE s.repo_id = NEW.repo_id AND s.snapshot_id <> NEW.snapshot_id
        ORDER BY s.computed_at DESC, s.snapshot_id DESC
        LIMIT 1);
```

```
    IF v_prev_level IS NOT NULL
       AND fn_risk_rank(NEW.risk_level) > fn_risk_rank(v_prev_level) THEN
        INSERT INTO risk_alert (repo_id, snapshot_id, previous_level, new_level)
        VALUES (NEW.repo_id, NEW.snapshot_id, v_prev_level, NEW.risk_level);
    END IF;
END
```

2.8 Application layer

REST API. A FastAPI [18] service exposes nine endpoints: a risk-ranked repository list with filtering and sorting, repository detail with contributor breakdown and risk reasons, snapshot history, dashboard summary counts, declining repositories (via the stored procedure), recent alerts, GitHub search, and adding or removing a tracked repository. Database connections come from a pool of five; every query is parameterised.

Dashboard. A React [19] single-page application (Figure 3.3) shows summary counts per risk level, the recent risk increases, and a searchable, filterable risk-ranked table. The detail page (Figure 3.4) shows the bus factor, the reasons for the risk level, the burnout signals, a bus-factor trend chart and the contributor breakdown.

Chapter 3

Results and Discussion

3.1 Experimental setup

The system was run on a Windows 11 laptop with MySQL Community Server 8.0.46, Python 3.12 and Node.js. The tracked list contained 43 public repositories: large, well-staffed projects (e.g. React, Kubernetes, CPython, Rust), popular single-maintainer utilities (e.g. `is-number`, `p-limit`, `chalk`), libraries known to be in maintenance mode or deprecated (`request`, `moment`) and one archived project (`jquery-mobile`). Snapshots were collected between 11 August and 16 October 2026; all figures below use each repository's latest snapshot from the full sync of 16 October 2026.

3.2 Data collected

One repository (`substack/node-mkdirp`) no longer exists on GitHub and returned HTTP 404; the remaining 42 were synchronised. Table 3.1 lists the resulting row counts.

Table 3.1: Rows stored per relation after the full sync.

Relation	Rows	Relation	Rows
repository	42	repo_release	3,145
contributor	17,306	weekly_activity	2,549
repo_contributor	10,052	bus_factor_snapshot	202
issue	6,211	risk_alert	9
pull_request	16,490		

Of the 6,211 issues, 1,510 have a recorded first-response time (only the 30 most recently updated issues with comments are looked up per sync).

3.3 Risk distribution

Figure 3.1 and Table 3.2 summarise the risk levels. **17 of the 42 repositories (40%) have a bus factor of 1**: a single person wrote at least half of all their commits. A further 4 have a

bus factor of 2. After the burnout signals are applied, 24 repositories are Critical, 4 High, none Medium and 14 Healthy.

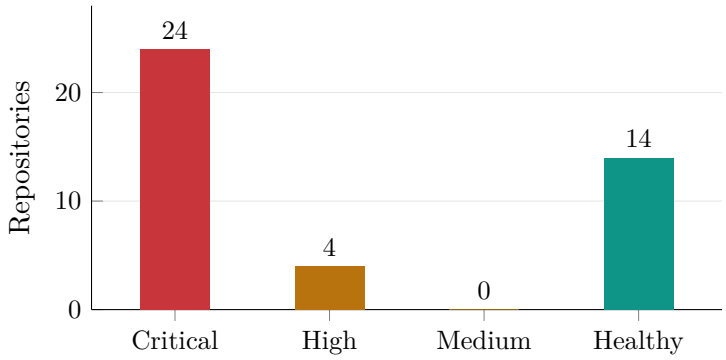


Figure 3.1: Risk level of the 42 repositories (latest snapshot).

Table 3.2: Risk levels with average bus factor and top-contributor share.

Risk level	Repositories	Avg. bus factor	Avg. top-contributor share
Critical	24	1.5	60.7%
High	4	3.3	28.9%
Medium	0	—	—
Healthy	14	14.3	15.0%

Figure 3.2 plots each repository's bus factor against the share of commits written by its top contributor. The two measures are strongly related: every repository whose top contributor wrote more than half of all commits has a bus factor of 1 by definition, and every Healthy repository has a top contributor below 40%. Popularity offers no protection: `scrcpy` (over 150,000 stars) has a bus factor of 1 with 90.9% of commits from one person, and `lodash` (over 61,000 stars) has 89.3%.

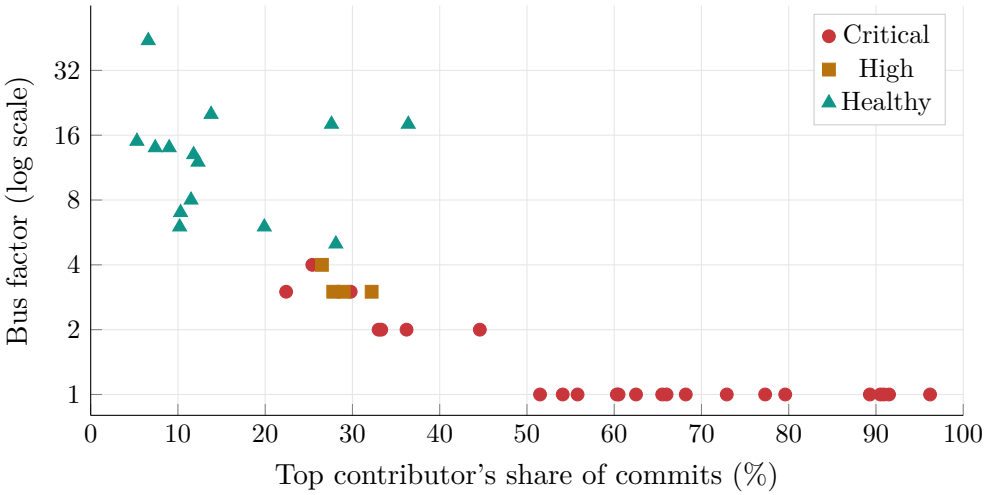


Figure 3.2: Bus factor versus top-contributor share for the 42 repositories.

Concentration is also uneven across ecosystems (Table 3.3). The 20 JavaScript repositories have an average bus factor of 2.8, against 9.9 for the 8 Python repositories, which reflects the

many small single-purpose npm packages in the sample. The sample per language is small and was not drawn randomly, so this is an observation about this data set, not a general claim.

Table 3.3: Average bus factor by primary language (languages with ≥ 2 repositories).

Language	Repositories	Avg. bus factor
JavaScript	20	2.8
Python	8	9.9
TypeScript	5	5.0
Go	2	15.0
C	2	3.0

3.4 Effect of the burnout signals

Across the 42 repositories the signals fired as follows: the top contributor was inactive for more than 180 days while holding at least 25% of commits in 14 repositories, commit activity fell by 50% or more in 5, the median issue response exceeded 30 days in 2, and 1 repository was archived.

When the burnout signals were introduced, the next sync raised the risk level of 9 repositories. The trigger recorded each change in `risk_alert` without any application code (Table 3.4). The same 9 repositories are returned by `CALL sp_declining_repos(1)`. These alerts reflect the richer scoring rather than a change in the projects themselves. From now on, alerts produced by the daily sync will reflect genuine changes.

Table 3.4: Risk increases recorded by `trg_snapshot_risk_alert`.

Repository	Before	After	Main driver
pallets/flask	High	Critical	top contributor inactive 2,294 days
tj/commander.js	High	Critical	activity −100% in last 12 weeks
caolan/async	High	Critical	top contributor inactive 775 days
psf/requests	Medium	Critical	activity −70%; top contributor inactive
gulpjs/gulp	Medium	Critical	activity −100%; top contributor inactive
jquery/jquery-mobile	Medium	Critical	repository archived
jashkenas/underscore	Medium	High	top contributor inactive 2,245 days
jquery/jquery	Medium	High	top contributor inactive 5,514 days
mochajs/mocha	Medium	High	top contributor inactive 4,604 days

3.5 Case studies and validation

Figure 3.4 shows the explanation produced for `pallets/flask`: a bus factor of 2 would normally be High, but the contributor who wrote a third of all commits last committed 2,294 days ago, so the level is raised to Critical. Several cases act as sanity checks on the method:

- **Known-healthy projects score Healthy.** React, Node.js, Rust, PyTorch, CPython, Kubernetes and VS Code all have bus factors of 6 or more and no dominant contributor.

- **Known-unmaintained projects score badly.** The archived `jquery-mobile` is Critical; `request` (deprecated by its maintainers in 2020) is High; `moment` (in maintenance mode) is Critical.
- **Calibration of the inactivity signal.** In a first version the inactivity signal fired whenever the top contributor had been silent for 180 days. This rated React as Medium, because its all-time top contributor, with only 10.2% of commits, had not committed for 264 days. In a project of that size such turnover is routine, so the signal now requires the inactive person to hold at least 25% of commits, and React is Healthy again.

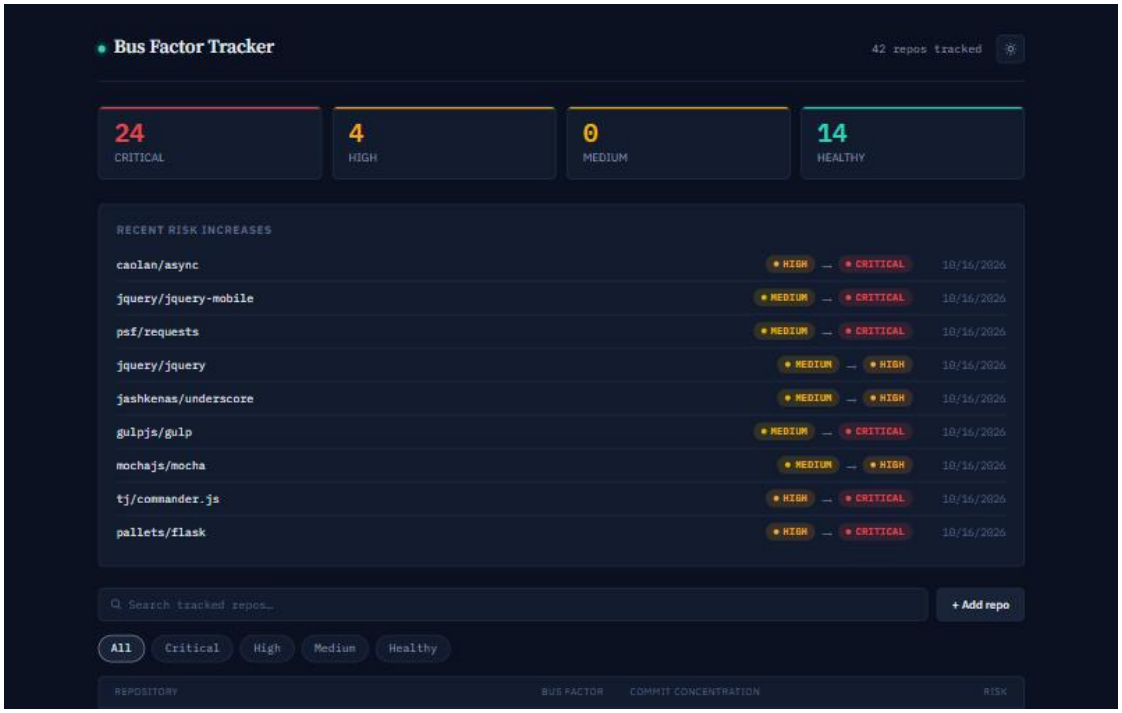


Figure 3.3: Dashboard: summary counts per risk level and the recent risk increases recorded by the trigger.

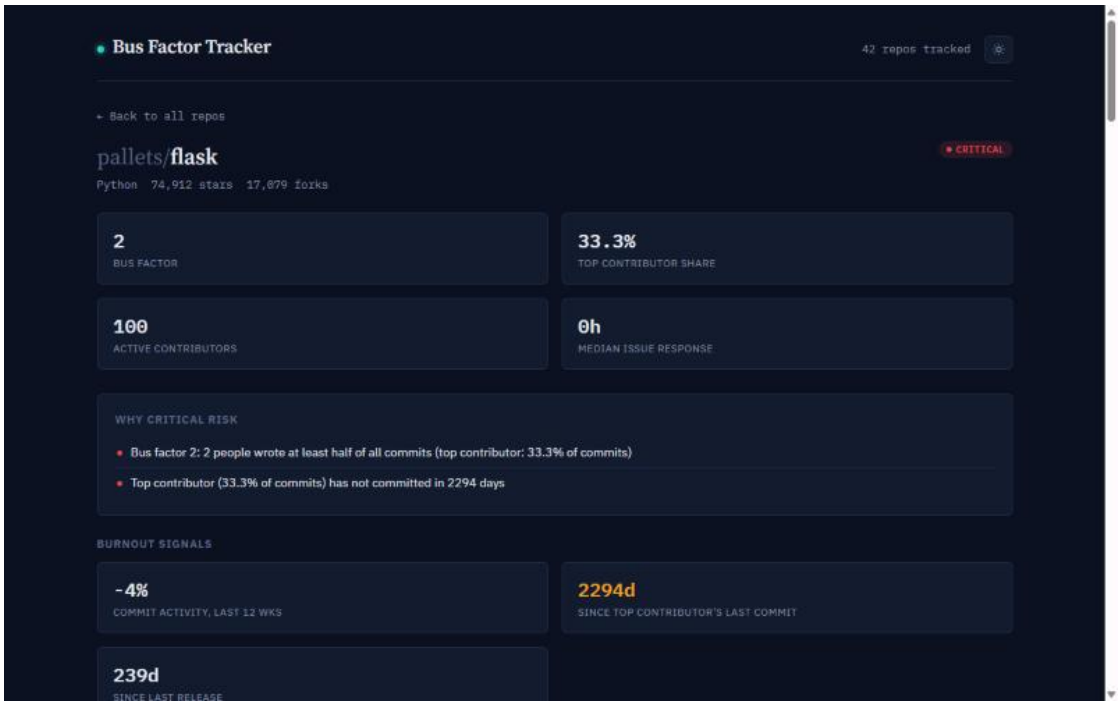


Figure 3.4: Repository detail for `pallets/flask`: the risk level with its reasons and the burnout signals.

3.6 Performance

Sync. The full sync of 43 repositories took 1,679.7 s (about 39 s per repository), dominated by GitHub API latency. During the run the engine waited on six `202 Accepted` responses, fell back once to deriving weekly totals from contributor statistics, and recovered from six dropped connections without losing any repository.

API. Table 3.5 shows the median latency of each read endpoint over 20 requests, measured in-process against the local database. The repository-detail endpoint originally took 186 ms for React (504 contributor rows) because its commit-share column used a correlated subquery that re-summed the repository's commits for every row. Replacing it with the window function `SUM(total_commits) OVER ()` computes the total once and reduced the latency to 8.6 ms, about 21 times faster, with identical results.

Table 3.5: Median API latency over 20 requests.

Endpoint	Median latency
`GET /api/repos` (risk-ranked list, via view)	5.6 ms
`GET /api/repos/facebook/react` (before optimisation)	186.3 ms
`GET /api/repos/facebook/react` (window function)	8.6 ms
`GET /api/repos/{owner}/{name}/trend`	3.8 ms
`GET /api/stats/summary`	4.3 ms
`GET /api/stats/declining` (stored procedure)	5.4 ms
`GET /api/alerts`	3.6 ms

3.7 Per-repository results

Table 3.6 lists the latest snapshot for every repository.

Table 3.6: Latest snapshot per repository. BF = bus factor; Top = top contributor's share of commits; ΔAct = commit-activity change; Inact. = days since the top contributor's last commit; "–" = not enough data.

Repository	Lang.	Stars	BF	Top %	ΔAct %	Inact.	Risk
Genymobile/scrcpy	C	152,325	1	90.9	−4.6	12	Critical
laravel/laravel	Blade	85,185	1	72.9	−49.3	12	Critical
expressjs/express	JS	69,572	1	65.5	15.9	4,625	Critical
lodash/lodash	JS	61,288	1	89.3	−84.1	117	Critical
vuejs/core	TS	54,648	1	54.1	7.0	264	Critical
Automattic/mongoose	JS	27,460	1	62.5	−18.1	5	Critical
chalk/chalk	JS	23,327	1	60.5	233.3	19	Critical
motdotla/dotenv	JS	20,547	1	79.6	73.8	5	Critical
sindresorhus/ky	TS	17,120	1	68.2	286.4	12	Critical
encode/httpx	Python	15,604	1	51.5	−100.0	397	Critical
BtbN/FFmpeg-Builds	Shell	11,831	1	90.5	128.2	19	Critical
benoitc/gunicorn	Python	10,701	1	60.3	2.3	40	Critical
cordiverse/cordis	TS	9,212	1	91.5	250.0	5	Critical
sindresorhus/execa	JS	7,619	1	66.0	417.2	292	Critical
sindresorhus/p-limit	JS	2,933	1	77.3	28.2	33	Critical
sindresorhus/is-online	JS	1,357	1	55.8	66.6	33	Critical
jonschlinkert/is-number	JS	286	1	96.2	–	3,029	Critical
pallets/flask	Python	74,912	2	33.3	−3.7	2,294	Critical
moment/moment	JS	47,895	2	33.0	10,900	971	Critical
tj/commander.js	JS	28,423	2	44.6	−100.0	145	Critical
caolan/async	JS	28,118	2	36.2	–	775	Critical
psf/requests	Python	54,461	3	29.8	−69.7	2,581	Critical
jquery/jquery-mobile†	JS	9,595	3	22.4	–	3,701	Critical
gulpjs/gulp	JS	32,921	4	25.4	−100.0	502	Critical
jashkenas/underscore	JS	27,312	3	32.2	−35.5	2,245	High
request/request	JS	25,489	3	29.0	–	2,441	High
mochajs/mocha	JS	22,902	3	27.8	27.1	4,604	High
jquery/jquery	JS	59,869	4	26.5	−30.1	5,514	High
FFmpeg/FFmpeg	C	65,277	5	28.1	22.2	5	Healthy
facebook/react	JS	251,163	6	10.2	−7.2	264	Healthy
pypa/pip	Python	10,293	6	19.9	−49.6	152	Healthy
microsoft/vscode	TS	193,749	7	10.3	23.6	173	Healthy
django/django	Python	91,401	8	11.5	−29.1	33	Healthy
golang/go	Go	139,284	12	12.3	19.5	54	Healthy
rails/rails	Ruby	58,841	13	11.8	38.7	12	Healthy
nodejs/node	JS	122,385	14	7.4	100.1	5	Healthy
python/cpython	Python	77,651	14	9.0	−2.1	180	Healthy
angular/angular	TS	101,062	15	5.3	−25.6	1,433	Healthy
tensorflow/tensorflow	C++	201,066	18	36.4	28.2	5	Healthy

Repository	Lang.	Stars	BF	Top %	ΔAct %	Inact.	Risk
kubernetes/kubernetes	Go	128,317	18	27.6	−31.6	124	Healthy
rust-lang/rust	Rust	120,415	20	13.8	17.9	5	Healthy
pytorch/pytorch	Python	104,804	44	6.6	16.3	5	Healthy

†Archived by its owner.

3.8 Discussion of limitations

Running the system on real data exposed several limitations, each of which points to a concrete improvement:

- **Automation bots count as contributors.** The top contributors of TensorFlow (`tensorflower-gardener`, 36.4%), Kubernetes (`k8s-ci-robot`, 27.6%) and PyTorch (`pytorchmergebot`) are bots that merge other people's work. Bots should be excluded before the bus factor is computed.
- **All-time history over-weights former maintainers.** `expressjs/express` is Critical because its original author wrote 65.5% of all commits but last committed 4,625 days ago, even though the project has other active maintainers. A recency-weighted bus factor (e.g. over the last 12 months) would capture current concentration better; the inactivity signal only partly compensates.
- **Commits are a proxy for effort.** Code review, issue triage, releases and documentation are not counted, and squash-merge workflows credit the person who merges.
- **Truncated contributor lists.** GitHub's contributor statistics cover the top 100 contributors per call. For very large projects the long tail is incomplete, and the stored count grows across syncs as different contributors enter the top 100.
- **Small baselines.** The activity change is unstable when the baseline is near zero: `moment` shows +10,900% because a few commits followed a long, almost silent period.
- **History starts at the first sync.** Trend analysis needs weeks of daily snapshots before `sp_declining_repos(180)` becomes meaningful.

3.9 Challenges and lessons learned

- **A query that silently returned nothing.** The first trend query joined snapshots on `s2.computed_at = s1.computed_at - INTERVAL 6 MONTH`. Snapshots are taken whenever the sync runs, so no two are exactly six months apart to the second, and the query always returned zero rows. It now selects the newest snapshot *at or before* the cut-off. Lesson: test every query on real data, not just for syntax.
- **Driver behaviour.** The installed MySQL connector returns stored-procedure results as dictionaries, which broke the API's tuple-based handling only once real rows existed. The endpoint now accepts both forms.

- **Porting from PostgreSQL to MySQL.** The design was first written for PostgreSQL. MySQL needed a different `INTERVAL` syntax, a window-function median instead of `PERCENTILE_CONT`, and a renamed `release` table.
- **Calibrating on real data.** The React case above showed that a signal that looks reasonable in isolation can mislabel large projects; thresholds must be checked against known cases.

Chapter 4

Conclusion and Future Work

4.1 Conclusion

This project built a working, database-centred system that estimates the bus factor and maintainer-burnout risk of real open-source repositories using only public GitHub data. The design applies core DBMS concepts where they solve real problems: a normalised schema in BCNF with cascading referential integrity, one ACID transaction per repository so that a failed sync never corrupts the current state, CTEs and window functions that compute the measurements inside the database, and a view, stored function, stored procedure and trigger that expose current risk, detect declines and record risk increases automatically.

On 42 widely used repositories the system showed that risk is widespread: 40% have a bus factor of 1, including projects with tens of thousands of stars. Adding burnout signals raised the risk of 9 repositories, and every change was captured by the trigger. Each risk level is explained in plain English. The API answers in under 10 ms, and a single window function made the slowest endpoint about 21 times faster.

4.2 Future work

- Filter automation bots and compute a recency-weighted bus factor.
- Count review, triage and release work, not only commits.
- Analyse a project's full dependency tree to report the riskiest dependencies of an application.
- Send e-mail or webhook notifications when a tracked repository becomes Critical.
- Validate the risk levels against a labelled set of projects that were later abandoned.

References

[1] Nadia Eghbal. *Roads and Bridges: The Unseen Labor Behind Our Digital Infrastructure*. Ford Foundation, 2016.

[2] Nadia Eghbal. *Working in Public: The Making and Maintenance of Open Source Software*. Stripe Press, 2020.

[3] npm, Inc. kik, left-pad, and npm. npm Blog, `https://blog.npmjs.org/post/141577284765/kik-left-pad-and-npm`, 2016. Published 23 March 2016.

[4] Jailton Coelho and Marco Tulio Valente. Why modern open source projects fail. In *Proceedings of the 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, pages 186–196, 2017.

[5] Naveen Raman, Minxuan Cao, Yulia Tsvetkov, Christian Kästner, and Bogdan Vasilescu. Stress and burnout in open source: Toward finding, understanding, and mitigating unhealthy interactions. In *Proceedings of the 42nd International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*, pages 57–60, 2020.

[6] Guilherme Avelino, Leonardo Passos, Andre Hora, and Marco Tulio Valente. A novel approach for estimating truck factors. In *Proceedings of the 24th IEEE International Conference on Program Comprehension (ICPC)*, pages 1–10, 2016.

[7] Valerio Cosentino, Javier Luis Cánovas Izquierdo, and Jordi Cabot. Assessing the bus factor of Git repositories. In *Proceedings of the 22nd IEEE International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, pages 499–503, 2015.

[8] Elgun Jabrayilzade, Mikhail Evtikhiev, Eray Tüzün, and Vladimir Kovalenko. Bus factor in practice. In *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 97–106, 2022.

[9] CHAOSS Project. Community health analytics in open source software. `https://chaoss.community/`, 2026. Accessed October 2026.

[10] Open Source Security Foundation. OpenSSF scorecard. `https://github.com/ossf/scorecard`, 2026. Accessed October 2026.

[11] GitHub, Inc. GitHub REST API documentation. `https://docs.github.com/en/rest`, 2026. Accessed October 2026.

[12] Peter Pin-Shan Chen. The entity-relationship model—toward a unified view of data. *ACM Transactions on Database Systems*, 1(1):9–36, 1976.

[13] Edgar F. Codd. A relational model of data for large shared data banks. *Communications of the ACM*, 13(6):377–387, 1970.

[14] Abraham Silberschatz, Henry F. Korth, and S. Sudarshan. *Database System Concepts*. McGraw-Hill, 7th edition, 2019.

[15] Ramez Elmasri and Shamkant B. Navathe. *Fundamentals of Database Systems*. Pearson, 7th edition, 2016.

[16] Theo Haerder and Andreas Reuter. Principles of transaction-oriented database recovery. *ACM Computing Surveys*, 15(4):287–317, 1983.

[17] Oracle Corporation. MySQL 8.0 reference manual. `https://dev.mysql.com/doc/refman/8.0/en/`, 2026. Accessed October 2026.

[18] Sebastián Ramírez. FastAPI documentation. `https://fastapi.tiangolo.com/`, 2026. Accessed October 2026.

[19] Meta Platforms, Inc. React documentation. `https://react.dev/`, 2026. Accessed October 2026.